

# MANUAL TÉCNICO



## VISIMAP

Técnico Superior de Desarrollo  
de Aplicaciones Web

Autor: Alberto Vera Mateos  
Curso Académico: 2025-2026

# Índice de Contenidos

|                                                                    |           |
|--------------------------------------------------------------------|-----------|
| <b>1. Introducción</b>                                             | <b>4</b>  |
| 1.1 Visimap                                                        | 4         |
| 1.2 Problemas detectados                                           | 4         |
| 1.3 Funcionalidades                                                | 5         |
| <b>2. Requisitos Funcionales y No Funcionales</b>                  | <b>6</b>  |
| 2.1 Requisitos funcionales                                         | 6         |
| 2.2 Requisitos no funcionales                                      | 7         |
| <b>3. Roles y Permisos</b>                                         | <b>9</b>  |
| 3.1 Roles                                                          | 9         |
| 3.2 Tabla de acciones por rol                                      | 9         |
| <b>4. Arquitectura</b>                                             | <b>11</b> |
| 4.1 Arquitectura Front-only                                        | 11        |
| 4.2 Frontend: React, TypeScript, Vite                              | 11        |
| 4.3 Frontend: React, TypeScript, Vite                              | 12        |
| 4.4 Por qué Supabase                                               | 14        |
| 4.5 Patrón Repository + Factory                                    | 15        |
| 4.6 Zustand y Rutas protegidas                                     | 16        |
| 4.7 Integración de Inteligencia Artificial                         | 16        |
| <b>5. Base de Datos</b>                                            | <b>18</b> |
| 5.1 Puntos clave de la base de datos                               | 18        |
| 5.2 Diagrama Entidad–Relación                                      | 18        |
| 5.3 Tablas del sistema                                             | 20        |
| 5.4 Vistas SQL para la IA                                          | 24        |
| 5.5 Storage y Edge Functions                                       | 25        |
| 5.6 Seguridad: Row Level Security                                  | 25        |
| <b>6. Estructura de Carpetas y Módulos</b>                         | <b>27</b> |
| <b>7. Instalación y Despliegue</b>                                 | <b>29</b> |
| 7.1 Requisitos previos                                             | 29        |
| 7.2 Instalación local                                              | 29        |
| 7.3 Variable de entorno                                            | 29        |
| 7.4 Comandos útiles                                                | 30        |
| 7.5 Despliegue en Vercel                                           | 30        |
| <b>8. Esquemas de Flujo</b>                                        | <b>31</b> |
| 8.1 Arquitectura general                                           | 31        |
| 8.2 Diagrama de secuencia – Flujo de autenticación                 | 31        |
| 8.3 Diagrama de secuencia – Registro de visita                     | 32        |
| 8.4 Diagrama de secuencia – Creación de evento en calendario       | 33        |
| 8.5 Diagrama de secuencia – Invitación de empleado (Edge Function) | 33        |
| 8.6 Diagrama de secuencia – Asistente IA                           | 34        |
| 8.7 Diagrama de secuencia – Exportación de histórico en PDF        | 35        |
| 8.8 Diagrama de secuencia – Sistema de Notas                       | 35        |

|                                                  |           |
|--------------------------------------------------|-----------|
| 8.9 Diagrama de secuencia – Notificaciones       | 36        |
| <b>9. Pruebas Realizadas</b>                     | <b>38</b> |
| 9.1 Pruebas funcionales                          | 38        |
| 9.2 Pruebas de Seguridad (Row level security)    | 40        |
| 9.3 Pruebas de Rendimiento                       | 40        |
| <b>10. Conclusión e Implementaciones Futuras</b> | <b>41</b> |

## CAPÍTULO 01 · Introducción

### 1.1 Visimap:

Visimap es una aplicación web de tipo SPA (Single Page Application). Su objetivo es digitalizar la gestión cotidiana del Museo MUVI de Villafranca de los Barros (Badajoz), sustituyendo los procesos manuales en papel por un sistema centralizado, escalable y seguro.

La plataforma permite registrar la afluencia de visitantes (individuales o por grupos), planificar el calendario de eventos del museo, administrar el personal con distintos niveles de privilegio, generar gráficos en tiempo real, exportar informes históricos en PDF y conversar con un asistente de Inteligencia Artificial que analiza los datos almacenados. Todo ello desde una interfaz responsive accesible desde cualquier dispositivo.

#### Propósito

Facilitar el control de acceso fluido y mejorar la toma de decisiones del personal del museo basándose en datos geográficos, organizativos y temporales, manteniendo una experiencia de usuario optimizada incluso en entornos de alta concurrencia.

### 1.2 Problemas detectados:

Antes de la implantación de Visimap, el museo MUVI gestionaba manualmente los registros de afluencia y eventos, lo que generaba un conjunto recurrente de problemas operativos:

| Problema                  | Solución que aporta Visimap                                                                                                                |
|---------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| Error humano y duplicidad | El registro manual en papel produce errores de conteo, datos duplicados y resultados inconsistentes que requieren revisión posterior.      |
| Retraso en el análisis    | Los informes en papel se demoran un día o más; con Visimap las estadísticas se generan en tiempo real.                                     |
| Falta de visualización    | En papel no se aprecian tendencias; Visimap permite ver de un vistazo qué meses, eventos o regiones aportan más afluencia.                 |
| Seguridad y privacidad    | Los registros físicos ocupan espacio, se deterioran y son vulnerables. Visimap centraliza todo en una base de datos con control de acceso. |
| Trazabilidad              | En papel no queda claro quién registró cada visita. Visimap asocia cada registro al empleado que lo creó.                                  |

## 1.3 Funcionalidades

El sistema implementa el siguiente conjunto de funcionalidades:

| Módulo                | Descripción funcional                                                                                                                                                                            |
|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Autenticación         | Inicio de sesión seguro con Supabase Auth, gestión de sesión persistente, recuperación de contraseña y flujo de invitaciones por correo electrónico para nuevos empleados.                       |
| Registro geográfico   | Mapa interactivo SVG de las provincias españolas y formulario lateral para visitantes internacionales. Permite registrar accesos individuales o por grupos.                                      |
| Calendario de eventos | Gestión completa (CRUD) de eventos del museo con visualización mensual, semanal y por lista. Permite asociar grupos de visitantes a cada evento.                                                 |
| Gestión de personal   | CRUD de empleados desde el panel de administrador, asignación de roles, control de estado (activo/inactivo) y eliminación de cuentas.                                                            |
| Dashboard analítico   | Gráficos en tiempo real generados con Recharts: distribución por procedencia, evolución temporal, ranking de eventos y picos de afluencia.                                                       |
| Histórico anual       | Vista nueva con totales mensuales y desgloses por provincia y país, exportable a PDF mediante jsPDF + jsPDF-Autotable.                                                                           |
| Asistente IA          | Página dedicada con chat conversacional basado en Groq (modelos Llama/Mixtral) que consulta la base de datos del museo y genera gráficos dinámicos en respuesta a preguntas en lenguaje natural. |
| Sistema de notas      | Notas internas tipo "post-it" que pueden asignarse a empleados concretos, con estados (normal, pendiente, finalizada) y fecha de actualización automática.                                       |
| Diseño responsive     | Interfaz mobile-first construida con Tailwind CSS, totalmente adaptable a tablets y móviles. Incluye modo claro y oscuro.                                                                        |

## CAPÍTULO 02 · Requisitos Funcionales y No Funcionales

---

En este capítulo se recogen los requisitos del sistema. Los requisitos funcionales describen qué debe hacer Visimap; los no funcionales describen cómo debe comportarse en términos de rendimiento, seguridad, usabilidad y mantenibilidad.

### 2.1 Requisitos funcionales

Se identifican con la nomenclatura RF1, RF2... Cada uno describe una capacidad de alto nivel del sistema, no un detalle de implementación.

- RF1:** El sistema permite el registro e inicio de sesión seguro de los usuarios mediante autenticación con Supabase Auth.
- RF2:** El sistema diferencia entre dos tipos de usuarios: administrador y trabajador, con permisos distintos asignados a cada rol.
- RF3:** El usuario puede registrar el número de visitantes seleccionando una provincia en un mapa interactivo SVG de España.
- RF4:** El sistema permite registrar visitantes internacionales seleccionando su país de origen desde un formulario lateral.
- RF5:** El sistema muestra estadísticas visuales mediante gráficos de barras, líneas y sectores generados con Recharts.
- RF6:** El usuario puede filtrar y consultar registros por provincia, país o fecha.
- RF7:** El sistema integra un calendario de eventos para anotar exposiciones, talleres, conferencias y otras actividades del museo.
- RF8:** El usuario puede realizar el CRUD completo (crear, editar y eliminar) de eventos del calendario según su rol.
- RF9:** El sistema permite asociar grupos de visitantes a cada evento, desglosados por provincia o país de procedencia.
- RF10:** El administrador puede realizar el CRUD completo de usuarios: invitarlos por email, modificarlos, activarlos/desactivarlos y eliminarlos.
- RF11:** El sistema dispone de un sistema de notas internas tipo post-it con CRUD, asignación a otro empleado y estados (normal, pendiente, finalizada).
- RF12:** El sistema integra un asistente conversacional de Inteligencia Artificial basado en Groq capaz de responder preguntas sobre los datos del museo en lenguaje natural.
- RF13:** El asistente IA puede generar gráficos dinámicos a petición del usuario y ejecutar consultas controladas contra la base de datos.
- RF14:** El sistema dispone de una vista de Histórico anual con totales mensuales y desgloses por provincia y país.
- RF15:** El sistema permite exportar estadísticas e informes históricos a formato PDF mediante jsPDF.

**RF16:** Los gráficos y el mapa se actualizan automáticamente cuando se añadan nuevos registros.

**RF17:** El sistema permite consultar el total acumulado de visitantes por provincia, país y fecha.

**RF18:** El usuario puede gestionar su perfil propio: cambiar nombre, avatar, contraseña y preferencias.

**RF19:** La interfaz dispone de modo claro y modo oscuro, con preferencia persistente entre sesiones.

## 2.2 Requisitos no funcionales

Se identifican con la nomenclatura RNF1, RNF2... y describen las cualidades transversales del sistema.

**RNF1. Rendimiento:** La aplicación debe cargar los datos y el mapa en menos de 3 segundos en una conexión media (4G).

**RNF2. Seguridad:** Uso de autenticación segura con JWT (gestionada por Supabase Auth) y políticas Row Level Security a nivel de base de datos.

**RNF3. Disponibilidad:** El sistema debe estar disponible al menos el 99% del tiempo gracias al hosting en Supabase y Vercel.

**RNF4. Usabilidad:** La interfaz será intuitiva, con una estructura clara, mensajes de validación visibles y fácil de utilizar tanto en escritorio como en tablet.

**RNF5. Escalabilidad:** El sistema podrá ampliarse fácilmente añadiendo nuevas provincias, gráficos, tipos de evento o roles sin reestructurar la base de datos.

**RNF6. Mantenibilidad:** El código estará escrito en TypeScript, documentado y dividido en componentes reutilizables de React, con un patrón Repository que aísla el acceso a datos.

**RNF7. Compatibilidad:** La aplicación funcionará correctamente en los navegadores Chrome, Firefox, Safari y Edge, así como en dispositivos móviles y tablets.

**RNF8. Accesibilidad:** Se aplicarán buenas prácticas de accesibilidad (etiquetas semánticas, contraste WCAG AA, navegación por teclado, primitivas Radix UI).

**RNF9. Privacidad:** Los datos de usuarios y registros estarán protegidos por RLS y no se compartirán con terceros.

**RNF10. Portabilidad:** Al estar desarrollada como SPA (Single Page Application), podrá desplegarse fácilmente en distintos entornos (local, staging, producción).

**RNF11. Tipado estático:** Todo el código fuente se desarrollará en TypeScript con strict mode activado, detectando errores en tiempo de compilación.

**RNF12. Despliegue continuo:** Cada push a la rama main disparará automáticamente un build y despliegue en Vercel, con previsualizaciones por Pull Request.

**RNF13. Diseño responsive:** La interfaz será mobile-first y se adaptará a cualquier resolución gracias a Tailwind CSS.

**RNF14. Trazabilidad:** Cada registro de visitante o evento mantendrá una referencia al usuario que lo creó para permitir auditorías posteriores.

**RNF15. *Open-source y portabilidad de proveedor:*** El uso del patrón Repository permite migrar de Supabase a otro BaaS reescribiendo solo la capa de datos, sin tocar páginas ni componentes.

## CAPÍTULO 03 · Roles y Permisos

### 3.1 Roles:

Como Visimap va dirigida a una institución pública como es el Museo MUVI de Villafranca de los Barros, los trabajadores son registrados por el administrador (en este caso el director del museo) para salvaguardar los datos de los registros de visitantes y evitar que sean manipulados por personas externas. Solamente los trabajadores del museo tienen acceso a ellos.

El sistema implementa control de acceso basado en roles con dos perfiles bien diferenciados, almacenados en la tabla roles y enlazados al perfil de cada empleado mediante el campo role\_id.

| Rol           | Descripción funcional                                                                                                                                                                                                                             |
|---------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Administrador | Perfil con control total del sistema. Es el único que puede gestionar al personal, acceder a las vistas analíticas (estadísticas e histórico anual), exportar informes y editar o eliminar registros ajenos. Coincide con la dirección del museo. |
| Trabajador    | Perfil operativo del personal del museo. Su función diaria es registrar visitantes, consultar y gestionar los eventos del museo y comunicarse con sus compañeros mediante el sistema de notas.                                                    |

### 3.2 Tabla de acciones por rol

A continuación se muestra la tabla de acciones que puede llevar a cabo cada rol dentro del sistema:

| Acción                                    | Trabajador | Administrador |
|-------------------------------------------|------------|---------------|
| Ver mapa interactivo                      | ✓          | ✓             |
| Registrar visitantes                      | ✓          | ✓             |
| Editar o eliminar registros de visitantes |            | ✓             |
| Ver calendario                            | ✓          | ✓             |
| Crear / editar / eliminar eventos         | ✓          | ✓             |
| Crear y gestionar notas internas          | ✓          | ✓             |
| Usar el asistente de IA                   |            | ✓             |
| Editar perfil propio                      | ✓          | ✓             |
| Ver estadísticas y gráficas analíticas    |            | ✓             |
| Acceder al histórico anual                |            | ✓             |
| Exportar informes y datos a PDF           |            | ✓             |

| Acción                                   | Trabajador | Administrador |
|------------------------------------------|------------|---------------|
| Gestionar usuarios (alta, baja, edición) |            | ✓             |
| Configurar el sistema                    |            | ✓             |

#### Cumplimiento desde la base de datos

Los permisos no se aplican únicamente en el frontend (que sería frágil): cada tabla del esquema PostgreSQL tiene políticas Row Level Security que **validan el rol del usuario autenticado** antes de devolver, modificar o borrar filas. Si un trabajador intentase acceder manipulando la URL, el frontend no le mostraría el menú y, aunque consiguiera invocar la API directamente, PostgreSQL le rechazaría la consulta.

## CAPÍTULO 04 · Arquitectura

### 4.1 Arquitectura Front-only

Visimap adopta una arquitectura Front-only, en la que toda la lógica de presentación reside en el navegador (cliente) y los servicios de persistencia, autenticación y APIs son delegados a un proveedor BaaS (Backend as a Service). Esta decisión arquitectónica elimina la necesidad de mantener servidores propios tipo Node/Express, simplifica enormemente el despliegue y minimiza los costes de infraestructura, todo ello sin renunciar a la seguridad gracias al sistema RLS (Row Level Security) de PostgreSQL.

| Capa              | Responsabilidad                                                          |
|-------------------|--------------------------------------------------------------------------|
| 1. Cliente        | React 19 + TypeScript renderiza la interfaz en el navegador del usuario. |
| 2. SDK Supabase   | La librería @supabase/supabase-js gestiona las llamadas REST y Auth.     |
| 3. PostgREST      | Supabase expone automáticamente la BD PostgreSQL como API RESTful.       |
| 4. PostgreSQL     | Base de datos relacional con RLS que valida cada petición.               |
| 5. Storage / Auth | Servicios complementarios para avatares y JWT de sesión.                 |
| 6. Edge Functions | Funciones serverless en Deno (invitación de usuarios).                   |

#### Ventaja clave

La separación clara entre presentación y datos permite que el frontend pueda evolucionar independientemente del backend. Si en el futuro fuese necesario migrar de Supabase a otro proveedor, solo tendrían que sustituirse las implementaciones concretas de los repositorios, sin tocar las páginas ni los componentes.

### 4.2 Frontend: React, TypeScript y Vite

| Tecnología | Versión | Función en el proyecto                                                                                |
|------------|---------|-------------------------------------------------------------------------------------------------------|
| React      | ^19.2.0 | Librería núcleo para construir interfaces basadas en componentes con renderizado declarativo y Hooks. |
| TypeScript | ^6.0.2  | Tipado estático que detecta errores en tiempo de compilación y mejora el autocompletado en el IDE.    |
| Vite       | ^7.2.4  | Empaquetador moderno con módulos ES nativos en desarrollo y HMR instantáneo.                          |

| Tecnología        | Versión   | Función en el proyecto                                           |
|-------------------|-----------|------------------------------------------------------------------|
| Tailwind CSS      | ^3.4.19   | Framework utility-first para los estilos de mi página.           |
| React Router      | ^7.13.0   | Enrutamiento con createBrowserRouter, rutas anidadas y guardias. |
| Zustand           | ^5.0.12   | Store global ligero (~1 KB) con persistencia en localStorage.    |
| shadcn/ui + Radix | ^1.x      | Sistema de componentes accesibles sobre primitivas Radix UI.     |
| Recharts          | ^3.8.1    | Librería de gráficos basada en SVG y D3.                         |
| FullCalendar      | ^6.1.20   | Calendario interactivo con vistas día/semana/mes/lista.          |
| Framer Motion     | ^12.23.26 | Animaciones declarativas en componentes.                         |
| Lucide React      | ^0.562.0  | Iconografía vectorial coherente y minimalista.                   |
| jsPDF + Autotable | ^4.2.1    | Generación de informes históricos exportables a PDF.             |
| html2canvas       | ^1.4.1    | Captura de elementos del DOM como imagen para inclusión en PDF.  |

### 4.3 Mapa Interactivo SVG de provincias

Uno de los componentes más singulares del frontend es el mapa interactivo de provincias españolas, construido íntegramente con SVG nativo y React, sin ninguna librería de cartografía externa. A continuación, se muestra la evolución visual del mapa desde su versión inicial hasta el resultado final con colores por comunidad autónoma, leyenda y estados interactivos.

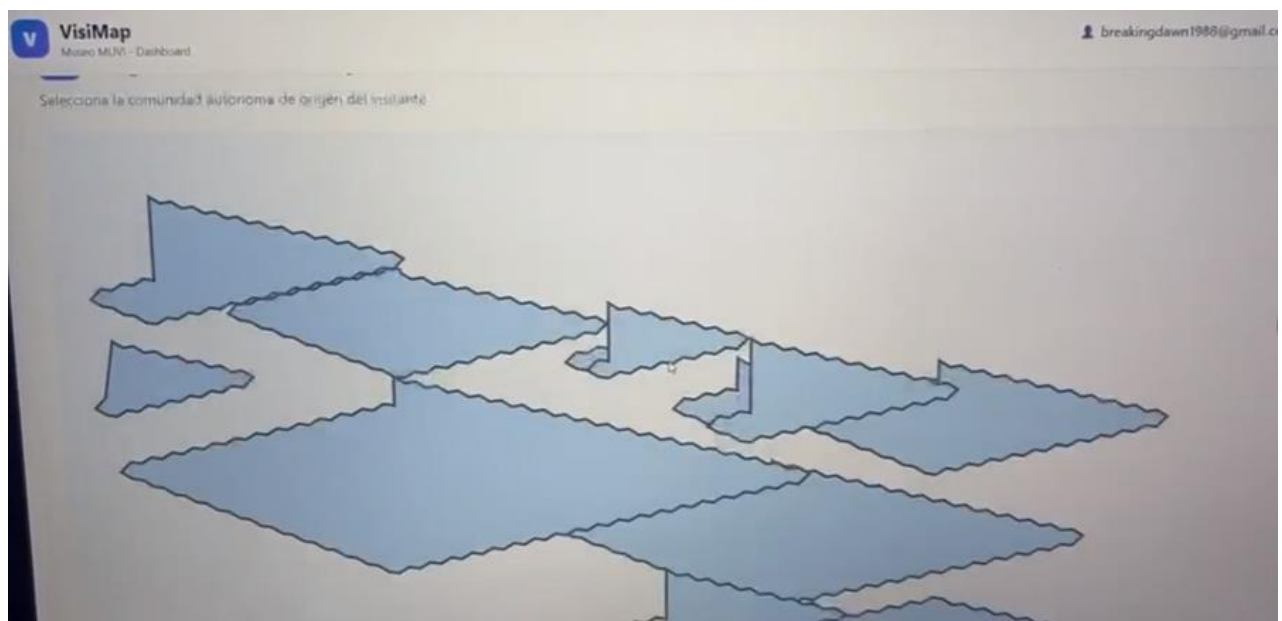


Figura 4.a — Mapa en su estado inicial, sin colores por comunidad



Figura 4.b — Mapa en su estado actual con colores por comunidad y leyenda

## Cómo se definen y pintan las provincias

Cada provincia del mapa es un elemento de trazado SVG cuya forma está descrita por una serie de coordenadas en píxeles. Todas estas formas están definidas en el archivo `src/utils/SpainProvinciasPaths.ts`, que exporta un objeto con una entrada por cada provincia. Cada entrada contiene el nombre visible (por ejemplo, “Badajoz”) y la cadena de trazado SVG con todos los puntos que forman su contorno (el path). Al renderizar el componente, React recorre ese objeto y genera un elemento SVG por provincia, colocándolos todos dentro de un mismo lienzo con el `viewBox` ajustado para encajar la geografía completa de España.

El color de relleno de cada provincia no está hardcodeado en el propio trazado, sino que se calcula en el momento del renderizado consultando el archivo `src/utils/ProvinciasColores.ts`. Este archivo mantiene dos tablas: la primera asocia cada comunidad autónoma a una clase de color de Tailwind CSS, y la segunda mapea cada una de las 52 provincias a la comunidad a la que pertenece. Gracias a esta separación, cambiar el color de toda una comunidad requiere modificar un único valor, y el cambio se aplica automáticamente a todas sus provincias sin tocar el componente visual. El propio componente del mapa (`src/components/app/SpainProvinciasMapa.tsx`) recorre los trazados, consulta el color correspondiente y, además, aplica en tiempo real un segundo estado de color cuando el usuario pasa el ratón por encima o selecciona una provincia para registrar una visita.

Sobre el mapa cada provincia presenta tres estados visuales distinguibles en tiempo real: el estado por defecto muestra el color de su comunidad, el estado de `hover` resalta la provincia en azul con una transición suave, y el estado de selección la marca en púrpura mientras abre el formulario de registro de visitantes.



Figura 4.b — Tooltip flotante con el nombre de la provincia

Un tooltip flotante muestra el nombre de la provincia siguiendo al cursor, y desaparece al salir del área. La leyenda lateral, construida dinámicamente a partir de la misma fuente de datos que el mapa, se adapta al tamaño de pantalla: oculta en móvil, visible al lateral en tableta y superpuesta al mapa en escritorio.

#### 4.4 Por qué Supabase:

La elección de Supabase como columna vertebral del backend no fue casual. Tras evaluar varias alternativas (Firebase, AWS Amplify, backend propio en Node + Express + PostgreSQL), Supabase resultó ser la opción que mejor encajaba con los objetivos académicos y técnicos del proyecto.

##### Razones técnicas

- PostgreSQL real, no NoSQL: a diferencia de Firebase, Supabase ofrece una base de datos relacional completa, con SQL estándar, joins, vistas, triggers y constraints. Esto permitía aplicar todo lo aprendido en los módulos de Bases de Datos del ciclo DAW.
- Row Level Security: PostgreSQL ofrece políticas RLS a nivel de fila, permitiendo construir un modelo de seguridad fino sin escribir middleware en el backend.
- API REST automática (PostgREST): cada tabla se convierte automáticamente en un endpoint REST con filtros, ordenación y paginación. Cero código boilerplate de servidor.
- Autenticación llave en mano: gestión de JWT, recuperación de contraseña, magic links e invitaciones por email sin escribir una línea de backend.
- Storage: almacenamiento de archivos (avatares) integrado con la misma autenticación.
- Edge Functions en Deno: posibilidad de ejecutar lógica privilegiada sin levantar un servidor propio.

## Razones para utilizarlo en mi proyecto

- Open source y self-hostable: si en algún momento fuese necesario, todo Supabase puede instalarse en un VPS propio con Docker. No hay vendor lock-in real.
- Gratuito: 500 MB de base de datos, 1 GB de storage y 50.000 MAU. Más que suficiente para el alcance del museo MUVI.
- Panel de administración visual: el SQL Editor, el Table Editor y el Auth panel facilitan enormemente la depuración durante el desarrollo.
- Documentación excelente y comunidad activa, fundamentales para este proyecto.

### Comparativa rápida

Firestore ofrecía mejor experiencia de tiempo real pero su modelo NoSQL no encajaba con los requisitos relacionales del museo. AWS Amplify era más potente pero excesivamente complejo y caro fuera del free tier. Un backend propio en Node hubiese sido excelente como ejercicio pero habría requerido el doble de tiempo de desarrollo. Supabase fue el equilibrio óptimo entre control técnico, productividad y coste cero.

## 4.5 Patrón Repository + Factory

Tras la primera tutoría con mi tutor Elias, me comentó el acceso a datos siguiendo el patrón Repository con Factory centralizada, obteniendo dos beneficios clave: desacoplar las páginas y componentes del proveedor concreto (Supabase), y facilitar la sustitución o el testeo mediante mocks en el futuro.

Cada entidad del dominio define una interfaz en `src/database/repositories/` que declara las operaciones permitidas. La implementación concreta para Supabase reside en `src/database/supabase/` y se obtiene siempre a través de `RepositoryFactory`:

| Repositorio         | Operaciones expuestas                                   |
|---------------------|---------------------------------------------------------|
| AuthRepository      | signIn, signOut, getSession, updatePassword, inviteUser |
| VisitorRepository   | CRUD de registro_visitante, lectura de pais y provincia |
| UserRepository      | CRUD de profiles, toggleStatus, getStats, uploadAvatar  |
| RoleRepository      | Lectura del catálogo de roles                           |
| EventRepository     | CRUD de evento y grupo_visitante (con relación)         |
| EventTypeRepository | Lectura de tipo_evento                                  |
| NotaRepository      | CRUD de notas con joins a perfiles                      |

## 4.6 Zustand y Rutas protegidas

El estado de autenticación es imprescindible en toda aplicación: cualquier ruta protegida, sidebar o componente puede necesitar saber si hay sesión activa, qué usuario está logueado o cuál es su rol. Se ha optado por usar Zustand.

Las rutas se definen mediante `createBrowserRouter` en `App.tsx`, organizadas en dos grupos protegidos por componentes guardia: `PublicRoute` (envuelve la landing y el login, redirige al dashboard si ya hay sesión) y `ProtectedRoute` (envuelve el dashboard, redirige a login si no hay sesión).

## 4.7 Integración de Inteligencia Artificial

Una de las funcionalidades más diferenciales de Visimap es el asistente conversacional que está en la página propia en `/dashboard/asistente`. Permite al personal del museo formular preguntas en lenguaje natural sobre los datos almacenados y recibir respuestas en formato texto y/o gráficos generados al vuelo.

### Proveedor: Groq

Tras evaluar varios modelos, se ha optado por Groq como motor del asistente. La elección responde a tres motivos principales:

- **Velocidad de respuesta:** la arquitectura LPU (Language Processing Unit) de Groq sirve modelos open-source con una latencia mucho menor que las APIs tradicionales, lo que mejora notablemente la experiencia conversacional del asistente y permite respuestas casi instantáneas.
- **Modelos open-source:** Groq sirve modelos abiertos como Llama 3 y Mixtral, lo que evita el vendor lock-in. Los mismos modelos pueden ejecutarse en otras plataformas (incluido self-hosting) si fuese necesario en el futuro.
- **Integración limpia** desde el navegador o desde una Edge Function mediante el SDK oficial `groq-sdk`, compatible con la especificación de la API de OpenAI.

| Componente       | Tecnología                           | Función                                                                                    |
|------------------|--------------------------------------|--------------------------------------------------------------------------------------------|
| Modelo           | Groq (Llama 3 / Mixtral)             | LLM que interpreta las preguntas, genera la respuesta y decide cuándo consultar la BBDD.   |
| SDK              | groq-sdk                             | Cliente oficial de Groq para llamar a la API de mensajes.                                  |
| Servicio interno | src/database/openai/openaiService.ts | Capa que construye el system prompt, parsea bloques especiales y orquesta la conversación. |

| Componente | Tecnología | Función                                                                               |
|------------|------------|---------------------------------------------------------------------------------------|
| Hook de UI | useChatIA  | Encapsula la lógica del chat (mensajes, streaming, sugerencias) para los componentes. |

## Flujo de funcionamiento

El servicio del asistente construye un sistema de prompt dinámico que pasa al modelo el esquema completo de la base de datos (tablas, columnas, relaciones), unas vistas para recortar tokens y un conjunto de instrucciones sobre cómo responder. Cuando el modelo necesita datos reales para contestar, los solicita en un bloque especial ```db-query``` que el cliente parsea y ejecuta de forma controlada contra Supabase. La respuesta vuelve al modelo, que produce el texto final y, opcionalmente, un bloque ```chart``` con la configuración de un gráfico de Recharts (más adelante se explica en un diagrama de secuencia).

- El usuario formula una pregunta en lenguaje natural ("¿Cuántos visitantes vinieron de Cáceres en julio?").
- El servicio envía la pregunta al modelo Groq con el sistema de prompt que incluye el esquema de la BBDD y las vistas.
- El modelo responde con un bloque `db-query` indicando la consulta SQL a ejecutar.
- El cliente ejecuta la consulta contra Supabase respetando las políticas RLS.
- El resultado se devuelve al modelo, que genera la respuesta final en texto natural y, si procede, una configuración de gráfico.
- La interfaz renderiza el texto y, si existe, dibuja el gráfico con Recharts.

### Consideración de seguridad

La API Key de Groq es sensible y no debe quedar expuesta indefinidamente en el navegador, por eso he puesto en el gitignore que no se suba el `.env` con las API Keys de supabase y la de Groq. Estas están en variables de entorno puestas en Vercel para que la aplicación se pueda desplegar.

## CAPÍTULO 05 · Base de Datos

### 5.1 Puntos clave de la base de datos

La base de datos de Visimap reside en PostgreSQL 15 alojado en Supabase. El diseño parte de cuatro premisas fundamentales que se aplican de forma transversal a todas las tablas:

- **Normalización (al menos 3FN):** no se duplican datos como nombres de provincias o tipos de evento; se almacenan en tablas catálogo y se referencian mediante claves foráneas.
- **Trazabilidad:** cada registro de visitante o evento mantiene una FK al usuario que lo creó (`id_usuario`), permitiendo auditar quién hizo qué y cuándo.
- **Seguridad por diseño:** cada tabla tiene políticas RLS activas que controlan qué filas puede leer, insertar, actualizar o eliminar cada rol.
- **Buenas prácticas de PostgreSQL:** usamos identificadores únicos (UUIDs) para los usuarios, igual que hace Supabase en `auth.users`. Las fechas se guardan con zona horaria (`TIMESTAMPTZ`) para evitar líos entre países. Y para campos con valores limitados (como `tipo_visita` o `estado_notas`), usamos reglas `CHECK` que solo permiten los valores válidos.

### 5.2 Diagrama Entidad–Relación

A continuación se muestran las relaciones principales entre tablas. Las flechas indican muchos→uno (la FK reside en el lado del muchos):

- **Modelo Relacional (MER):**

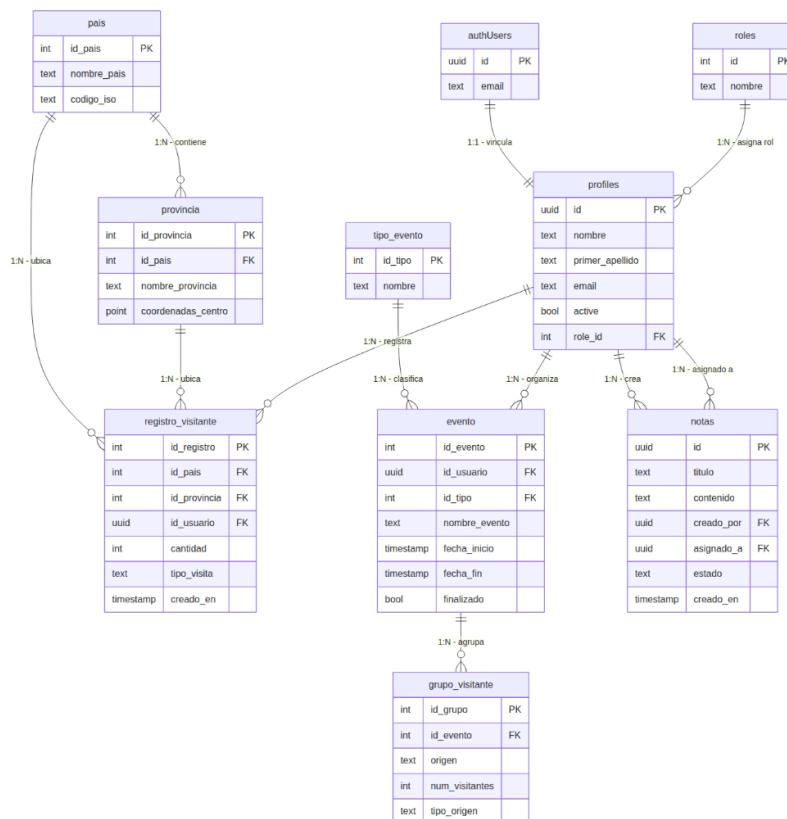


Figura 5.a — MER de la base de datos de Visimap

- **Base de datos en Supabase:**

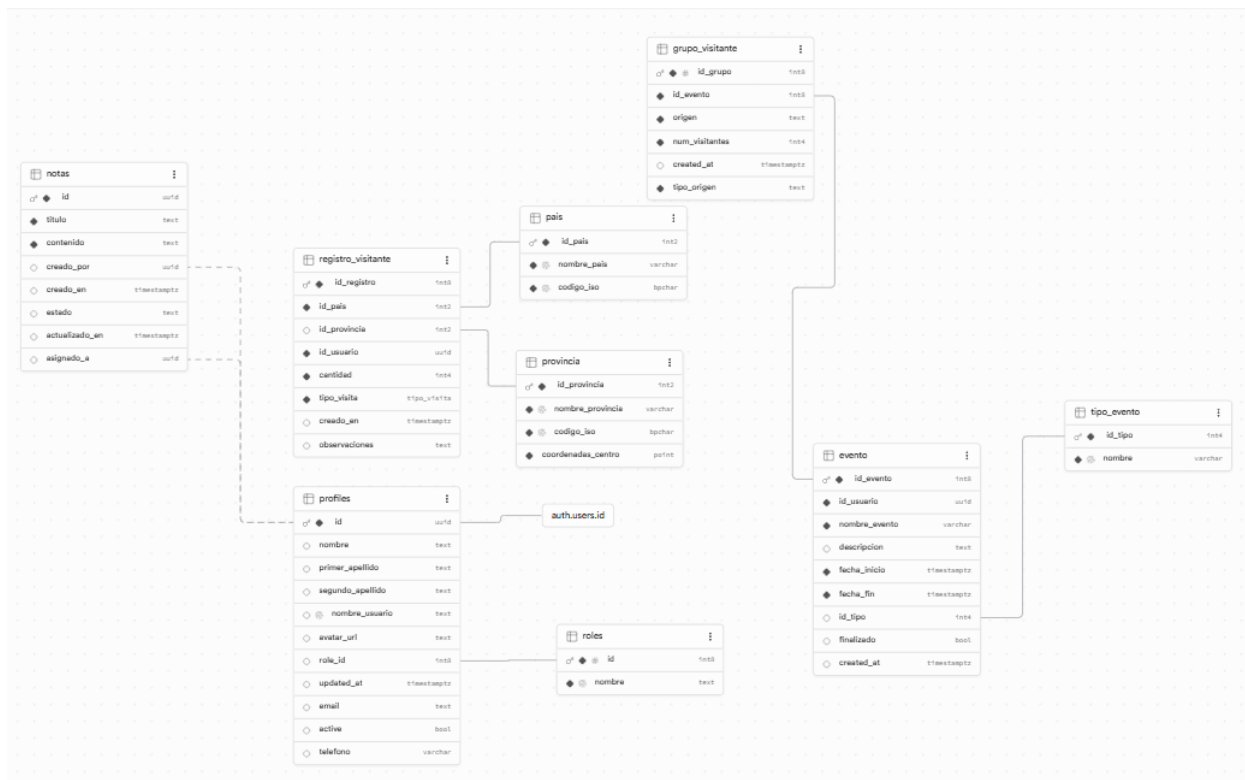


Figura 5.b — Diagrama de la base de datos de Visimap de Supabase

## 5.3 Tablas del sistema

A continuación, voy a explicar cada tabla que compone el modelo, indicando su esquema de columnas, restricciones y la justificación funcional de su existencia.

### Tabla: **profiles**

**Propósito:** almacena el perfil extendido de cada usuario del sistema. Supabase Auth ya gestiona credenciales y JWT en su tabla interna `auth.users`, pero esa tabla solo guarda email y metadatos. `profiles` añade datos del dominio: nombre, apellidos, teléfono, rol corporativo, avatar y estado activo/inactivo. Se relaciona 1:1 con `auth.users` mediante el mismo UUID en la columna `id`.

| Columna                       | Tipo        | Restricciones                     | Descripción                                              |
|-------------------------------|-------------|-----------------------------------|----------------------------------------------------------|
| <code>id</code>               | UUID        | PK, FK→ <code>auth.users</code>   | Identificador del usuario.                               |
| <code>nombre_usuario</code>   | TEXT        | UNIQUE NOT NULL                   | Nombre de usuario único, visible en notas.               |
| <code>nombre</code>           | TEXT        | NOT NULL                          | Nombre de pila.                                          |
| <code>primer_apellido</code>  | TEXT        | NOT NULL                          | Primer apellido.                                         |
| <code>segundo_apellido</code> | TEXT        | NULL                              | Segundo apellido (opcional).                             |
| <code>email</code>            | TEXT        | UNIQUE NOT NULL                   | Correo electrónico (espejo de <code>auth.users</code> ). |
| <code>telefono</code>         | TEXT        | NULL                              | Teléfono de contacto.                                    |
| <code>role_id</code>          | INT         | FK→ <code>roles</code> , NOT NULL | Rol corporativo (admin / trabajador).                    |
| <code>avatar_url</code>       | TEXT        | NULL                              | URL pública de la foto de perfil en Storage.             |
| <code>active</code>           | BOOL        | DEFAULT true                      | Si está activo. Los inactivos no pueden hacer login.     |
| <code>created_at</code>       | TIMESTAMPTZ | DEFAULT now()                     | Fecha de alta.                                           |
| <code>updated_at</code>       | TIMESTAMPTZ | DEFAULT now()                     | Última modificación.                                     |

### Tabla: roles

**Propósito:** catálogo de roles corporativos disponibles en el sistema. Se almacena en una tabla independiente (en lugar de un ENUM o un campo TEXT) por tres motivos:

1. Permite añadir nuevos roles sin migrar el esquema
2. Hace cumplir la integridad referencial automáticamente
3. Permite añadir metadatos al rol (descripción, fecha de creación) en el futuro.

| Columna     | Tipo        | Restricciones   | Descripción                             |
|-------------|-------------|-----------------|-----------------------------------------|
| id          | INT         | PK SERIAL       | Identificador autoincremental.          |
| nombre      | TEXT        | UNIQUE NOT NULL | Nombre del rol: "admin" o "trabajador". |
| descripcion | TEXT        | NULL            | Descripción funcional del rol.          |
| created_at  | TIMESTAMPTZ | DEFAULT now()   | Fecha de creación.                      |

### Tabla: pais

**Propósito:** catálogo precargado de los países del mundo. Necesario para registrar visitantes que provienen de fuera de España o que, aun siendo españoles, se identifican por país sin desglose provincial. Almacenar el identificador en lugar del nombre evita inconsistencias ortográficas (p. ej. "Reino Unido" vs "Inglaterra" vs "UK").

| Columna     | Tipo    | Restricciones   | Descripción                                |
|-------------|---------|-----------------|--------------------------------------------|
| id_pais     | INT     | PK SERIAL       | Identificador del país.                    |
| nombre_pais | TEXT    | UNIQUE NOT NULL | Nombre oficial en español.                 |
| codigo_iso  | TEXT(2) | UNIQUE          | Código ISO 3166-1 alpha-2 (ES, FR, US...). |

### Tabla: provincia

**Propósito:** catálogo de las 50 provincias españolas más Ceuta y Melilla. Se separa de país porque el mapa interactivo del frontend solo permite desglose provincial cuando el país es España. El campo id\_provincia de registro\_visitante es nullable: para visitantes internacionales solo se rellena id\_pais.

| Columna          | Tipo | Restricciones   | Descripción                           |
|------------------|------|-----------------|---------------------------------------|
| id_provincia     | INT  | PK SERIAL       | Identificador de la provincia.        |
| nombre_provincia | TEXT | UNIQUE NOT NULL | Nombre oficial: Madrid, Badajoz, etc. |

### Tabla: registro\_visitante

**Propósito:** tabla transaccional principal del sistema. Cada fila representa una entrada de visitantes realizada por un empleado en un momento concreto. Soporta tanto visitas individuales como pequeños grupos espontáneos (familias, parejas) que llegan a la ventanilla sin pertenecer a un evento programado. Para los grupos asociados a eventos se utiliza grupo\_visitante.

| Columna       | Tipo        | Restricciones                             | Descripción                      |
|---------------|-------------|-------------------------------------------|----------------------------------|
| id_registro   | BIGSERIAL   | PK                                        | Identificador del registro.      |
| id_pais       | INT         | FK→pais, NOT NULL                         | País del visitante.              |
| id_provincia  | INT         | FK→provincia, NULL                        | Provincia (solo si país=España). |
| id_usuario    | UUID        | FK→profiles, NOT NULL                     | Empleado que creó el registro.   |
| cantidad      | INT         | CHECK > 0                                 | Número de personas registradas.  |
| tipo_visita   | TEXT        | CHECK IN ('individual','familia','grupo') | Categoría del registro.          |
| observaciones | TEXT        | NULL                                      | Notas opcionales del empleado.   |
| creado_en     | TIMESTAMPTZ | DEFAULT now()                             | Momento exacto del registro.     |

### Tabla: evento

**Propósito:** almacena los eventos programados del museo (exposiciones, talleres, conferencias, bodas, mantenimientos...). Cada evento tiene un rango temporal (fecha\_inicio, fecha\_fin), un tipo categorizado, un creador y un bandera finalizado que se marca cuando el evento termina, lo que permite separar lo planificado de lo histórico en el calendario.

| Columna       | Tipo        | Restricciones            | Descripción                        |
|---------------|-------------|--------------------------|------------------------------------|
| id_evento     | BIGSERIAL   | PK                       | Identificador del evento.          |
| nombre_evento | TEXT        | NOT NULL                 | Título visible en el calendario.   |
| descripcion   | TEXT        | NULL                     | Descripción larga del evento.      |
| fecha_inicio  | TIMESTAMPTZ | NOT NULL                 | Inicio del evento.                 |
| fecha_fin     | TIMESTAMPTZ | CHECK ≥ fecha_inicio     | Fin del evento.                    |
| id_tipo       | INT         | FK→tipo_evento, NOT NULL | Categoría (taller, exposición...). |
| id_usuario    | UUID        | FK→profiles, NOT NULL    | Empleado que creó el evento.       |
| finalizado    | BOOL        | DEFAULT false            | Marcado al completarse el evento.  |
| created_at    | TIMESTAMPTZ | DEFAULT now()            | Fecha de creación del registro.    |

### Tabla: tipo\_evento

**Propósito:** catálogo cerrado y editable de tipologías de evento. Permite al frontend asignar colores coherentes a cada tipo en el calendario (p. ej. exposiciones en violeta, talleres en verde, bodas en rosa) sin hardcodear nombres en el código. El mapeo de colores se gestiona desde src/constantes/.

| Columna | Tipo | Restricciones   | Descripción                                                                   |
|---------|------|-----------------|-------------------------------------------------------------------------------|
| id_tipo | INT  | PK SERIAL       | Identificador del tipo.                                                       |
| nombre  | TEXT | UNIQUE NOT NULL | Etiqueta: visita, exposicion, taller, conferencia, mantenimiento, boda, otro. |

### Tabla: grupo\_visitante

**Propósito:** registra los grupos de visitantes asociados a un evento. A diferencia de registro\_visitante (visitas espontáneas a la ventanilla), grupo\_visitante documenta los grupos que asisten a un evento programado, desglosados por procedencia. Por ejemplo, una visita escolar de Cáceres puede tener una fila con tipo\_origen='provincia', origen='Cáceres', num\_visitantes=25.

| Columna        | Tipo        | Restricciones                 | Descripción                               |
|----------------|-------------|-------------------------------|-------------------------------------------|
| id_grupo       | BIGSERIAL   | PK                            | Identificador del grupo.                  |
| id_evento      | BIGINT      | FK→evento, ON DELETE CASCADE  | Evento al que pertenece.                  |
| tipo_origen    | TEXT        | CHECK IN ('provincia','pais') | Si el origen es provincia o país.         |
| origen         | TEXT        | NOT NULL                      | Nombre del origen (texto libre validado). |
| num_visitantes | INT         | CHECK > 0                     | Tamaño del grupo.                         |
| created_at     | TIMESTAMPTZ | DEFAULT now()                 | Momento del registro.                     |

### Tabla: notas

**Propósito:** sistema de comunicación interna entre empleados al estilo "post-it digital". Cualquier empleado puede crear una nota, opcionalmente asignársela a otro compañero (campo asignado\_a) y darle un estado: normal, pendiente o finalizada. Las notas finalizadas hace más de 7 días se ocultan automáticamente del listado para no acumular ruido (lógica implementada en el repositorio, no en SQL).

| Columna   | Tipo | Restricciones                | Descripción                      |
|-----------|------|------------------------------|----------------------------------|
| id        | UUID | PK DEFAULT gen_random_uuid() | Identificador único.             |
| titulo    | TEXT | NOT NULL                     | Título de la nota.               |
| contenido | TEXT | NOT NULL                     | Cuerpo de la nota (texto plano). |

| Columna        | Tipo        | Restricciones                                | Descripción                        |
|----------------|-------------|----------------------------------------------|------------------------------------|
| creado_por     | UUID        | FK→profiles, NOT NULL                        | Empleado que la creó.              |
| asignado_a     | UUID        | FK→profiles, NULL                            | Empleado destinatario (opcional).  |
| estado         | TEXT        | CHECK IN ('normal','pendiente','finalizada') | Estado de la tarea.                |
| creado_en      | TIMESTAMPTZ | DEFAULT now()                                | Fecha de creación.                 |
| actualizado_en | TIMESTAMPTZ | NULL                                         | Fecha del último cambio de estado. |

## 5.4 Vistas SQL para la IA

Visimap define tres vistas SQL en PostgreSQL que actúan como capa de abstracción entre el modelo relacional físico y los consumidores de datos (asistente IA, dashboard analítico e histórico anual). Las vistas unifican los flujos de visitantes que provienen de la ventanilla (registro\_visitante) y los grupos asociados a eventos (grupo\_visitante), enriquecen los registros con datos de procedencia y empleado, y precálculan agregados diarios. De este modo el frontend y la IA consultan una única fuente sin tener que construir UNIONs ni JOINs en cada gráfico, simplificando enormemente las consultas y reduciendo la cantidad de tokens necesarios en los prompts del asistente.

### Vista: vista\_visitantes\_totales

**Propósito:** vista de unificación más sencilla. Combina mediante UNION ALL las dos tablas de visitantes en una estructura común con cuatro columnas: origen ('ventanilla' o 'evento'), id\_origen (id del registro o grupo), total\_personas y fecha. Es la fuente principal de los gráficos del dashboard analítico y del histórico anual, ya que permite contar visitantes totales sin importar si vinieron espontáneamente o asociados a un evento.

### Vista: vista\_visitas\_unificadas

**Propósito:** vista enriquecida pensada para el asistente IA. Además de unificar las dos tablas de visitantes, resuelve los joins con provincia, país y profiles para devolver el nombre legible de la procedencia (provincia o país) y el nombre del trabajador que registró la visita. Incluye también una columna tipo\_origen ('provincia' o 'país') que indica qué tipo de procedencia tiene cada fila. Su existencia evita que la IA tenga que pedir múltiples consultas encadenadas: con una sola SELECT obtiene fecha, cantidad, tipo de entrada, procedencia legible y autor.

### Vista: vista\_visitas\_agrupadas\_diarias

**Propósito:** vista agregada construida sobre la anterior. Aplica un GROUP BY por fecha, tipo\_entrada y nombre\_trabajador, devolviendo la suma de personas y el número de registros por cada combinación. Es el insumo ideal para preguntas analíticas frecuentes ('cuántas visitas registró cada

empleado en agosto', 'cuál fue el día con más afluencia') sin que la IA tenga que calcular agregados sobre filas individuales.

Las tres vistas se han creado con la opción `security_invoker = true`, lo que significa que se ejecutan con los permisos del usuario que las consulta (no con los del creador de la vista). Así, las políticas Row Level Security definidas sobre las tablas base se siguen aplicando correctamente cuando el frontend o la IA leen a través de las vistas.

## 5.5 Storage y Edge Functions

### Bucket de Storage: avatars

Supabase Storage gestiona los archivos binarios. Visimap utiliza un único bucket público llamado avatars para almacenar las fotografías de perfil de los empleados. La subida se realiza desde `SupabaseUserRepository.uploadAvatar()`, que genera un nombre de archivo único (`userId-random.ext`), lo sube y devuelve la URL pública para guardarla en `profiles.avatar_url`.

### Edge Function: invite-user

Cuando un administrador crea un nuevo empleado, no se le asigna contraseña manualmente; en su lugar se invoca la Edge Function `invite-user`, que utiliza la API privilegiada de Supabase Admin (con la `service_role` key almacenada como secreto en el servidor) para enviar un correo de invitación. El empleado recibe un enlace mágico que le lleva a la pantalla `EstablecerContraseña`, donde define su clave por primera vez.

#### Por qué Edge Function y no llamada directa

La operación de invitar requiere la `service_role` key, que jamás debe estar en el navegador. Al delegar la invitación a una Edge Function, la clave permanece como secreto en el entorno serverless de Supabase y solo los admins autenticados (validados por la propia función) pueden invocarla.

## 5.6 Seguridad: Row Level Security

Row Level Security es la característica más potente de PostgreSQL para seguridad fina y la columna vertebral del modelo de seguridad de Visimap. Cada tabla tiene RLS enabled y un conjunto de políticas que filtran qué filas puede ver, insertar, modificar o eliminar cada usuario en función de su rol y de la propia fila.

| Tabla              | Operación       | Política aplicada                                               |
|--------------------|-----------------|-----------------------------------------------------------------|
| profiles           | SELECT          | Cualquier usuario autenticado.                                  |
| profiles           | UPDATE / DELETE | Solo el propio usuario o usuarios con <code>role=admin</code> . |
| registro_visitante | SELECT          | Todos los empleados autenticados.                               |
| registro_visitante | INSERT          | Cualquier empleado activo ( <code>active=true</code> ).         |

| Tabla                               | Operación            | Política aplicada                            |
|-------------------------------------|----------------------|----------------------------------------------|
| registro_visitante                  | UPDATE / DELETE      | Creador del registro o admin.                |
| evento, grupo_visitante             | SELECT / INSERT      | Todos los empleados.                         |
| evento, grupo_visitante             | UPDATE / DELETE      | Creador o admin.                             |
| notas                               | SELECT               | Creador, asignado o admin.                   |
| notas                               | UPDATE               | Creador o asignado (cambiar estado).         |
| roles, pais, provincia, tipo_evento | SELECT               | Todos los autenticados (catálogos públicos). |
| roles, pais, provincia, tipo_evento | INSERT/UPDATE/DELETE | Solo admin.                                  |

### El usuario "anónimo"

La anon key que usa el frontend solo permite hablar con la API; las políticas RLS exigen un JWT firmado por Supabase Auth con un usuario válido. Si alguien interceptase la anon key y la usase desde otro cliente, no podría leer ni una sola fila sin antes autenticarse con un usuario real activo. Adicionalmente, el frontend implementa una doble verificación: tras hacer login, el método `fetchProfile()` en `App.tsx` comprueba si `profiles.active === false` y, en ese caso, fuerza un `signOut()`.

## CAPÍTULO 06 · Estructura de Carpetas y Módulos

Visimap está organizado según los criterios y recomendaciones vistas en clase, separando la parte de presentación, lógica de datos, interfaces, rutas, etc .

```
visimap/
├── .github/workflows/    # Automatización y CI/CD (Despliegue automático de TypeDoc)
├── docs/                 # Documentación técnica y manuales corporativos
├── public/               # Assets estáticos y recursos públicos
├── src/
│   ├── assets/           # Recursos multimedia (imágenes, logos, SVGs)
│   ├── components/       # Componentes React (divididos en app/ y ui/)
│   ├── constantes/       # Constantes y valores globales del sistema
│   ├── database/         # Servicios de datos, repositorios y config de Supabase/OpenAI
│   ├── hooks/            # Hooks personalizados de React
│   ├── interfaces/       # Definiciones de tipos e interfaces TypeScript
│   ├── layouts/          # Plantillas de diseño y envoltorios de página
│   ├── pages/            # Componentes de página (vistas principales)
│   ├── rutas/            # Configuración y definición de rutas (React Router)
│   ├── stores/           # Gestión de estado global con Zustand
│   ├── styles/           # Configuración de estilos globales y Tailwind
│   ├── utils/            # Funciones de utilidad y lógica compartida
│   ├── App.tsx           # Componente raíz y configuración de proveedores
│   └── main.tsx          # Punto de entrada principal de la aplicación
├── .env                  # Variables de entorno (no incluir en control de versiones)
├── index.html            # Plantilla HTML de entrada para Vite
├── package.json          # Dependencias y scripts del proyecto
├── tailwind.config.js    # Configuración de los tokens de diseño de Tailwind
├── tsconfig.json         # Configuración base de TypeScript
├── vercel.json           # Configuración para el despliegue en Vercel
└── vite.config.ts        # Configuración del entorno de compilación de Vite
```

Figura 6.a — Organización de las carpetas de Visimap

Cada carpeta de src/ tiene un propósito único y bien definido.

| Carpeta                     | Contenido y propósito                                                                                                                           |
|-----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| src/assets                  | Imágenes WebP optimizadas: logos (claro y oscuro), fondos de login, fotos del museo MUVI.                                                       |
| src/components/ui           | Primitivas shadcn/ui: button, input, card, sidebar, breadcrumb, tooltip, etc.                                                                   |
| src/components/app          | Componentes específicos del dominio: SpainProvinciasMapa, Calendario, FormularioRegistroEquipo, TablaUsuarios, NotificationBell, ThemeToggle... |
| src/components/app/graficos | Subsistema de gráficos y chat IA: GraficosPanel, TarjetaGrafico, ChatIA.                                                                        |
| src/components/app/modales  | Modales reutilizables (confirmación, edición, evento).                                                                                          |

| Carpeta                           | Contenido y propósito                                                                                                                                |
|-----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| src/components/app/notas          | NotaCard y NotaModal del sistema de comunicación.                                                                                                    |
| src/constantes                    | Constantes globales (provincias, mapeos de colores).                                                                                                 |
| src/database/repositories         | Interfaces de los repositorios (contratos del dominio).                                                                                              |
| src/database/supabase             | Implementaciones concretas de los repositorios contra Supabase.                                                                                      |
| src/database/openai               | Cliente y servicio de Claude (Anthropic) para el asistente IA. (La carpeta conserva el nombre histórico, pero internamente usa el SDK de Anthropic). |
| src/database/RepositoryFactory.ts | Factoría centralizada de instancias singleton de repositorios.                                                                                       |
| src/hooks                         | Hooks personalizados: useChatIA, use-mobile.                                                                                                         |
| src/interfaces                    | Tipos e interfaces TypeScript del dominio: Auth, Perfil, Visitor, Evento, Nota, Rol, ChatIA, Graficos, Historial.                                    |
| src/layouts                       | DashboardLayout (sidebar + header + outlet).                                                                                                         |
| src/pages                         | Vistas principales: LandingPage, MapaVisitantes, VistaUsuarios, VistaCalendario, Graficos, Asistente, Historial, Notas, VistaPerfil.                 |
| src/rutas                         | Guardias ProtectedRoute / PublicRoute.                                                                                                               |
| src/stores                        | Stores Zustand (authStore con persistencia en localStorage).                                                                                         |
| src/styles                        | Estilos globales y tokens de diseño.                                                                                                                 |
| src/utils                         | Funciones de apoyo: ProvinciasColores, SpainProvinciasPaths, helpers.                                                                                |

### Alias de importación

Vite y TypeScript están configurados con el alias `@/` apuntando a `src/`, lo que permite escribir `import { useAuthStore } from "@/stores/authStore"` desde cualquier punto del proyecto, sin lidiar con rutas relativas largas.

### Anotación

Dentro de la carpeta docs, está en local una carpeta API donde tenemos toda la información sobre el TypeDocs. Para poder visualizarla, he creado en docs un `index.html` donde se puede visualizar la documentación creada con TypeDocs y luego a parte podemos consultar los manuales técnicos, de usuario y el de despliegue de Visimap. También puedes verlo en mi github, abajo en el Readme, hay un botón que pone ver documentación técnica, donde lo pulsas y te lleva al mismo sitio.

## CAPÍTULO 07 · Instalación y Despliegue

### 7.1 Requisitos previos

| Herramienta      | Versión                       | Función                                                 |
|------------------|-------------------------------|---------------------------------------------------------|
| Node.js          | ≥ 18 LTS (recomendado 20 LTS) | Runtime JavaScript para Vite y npm.                     |
| npm              | ≥ 9                           | Gestor de paquetes (incluido en Node).                  |
| Git              | Cualquier versión moderna     | Para clonar el repositorio.                             |
| Cuenta Supabase  | —                             | Proyecto creado en supabase.com.                        |
| Cuenta Anthropic | —                             | Para el chat IA con Claude (clave de API).              |
| Cuenta Vercel    | —                             | Para el despliegue del frontend.                        |
| IDE recomendado  | VS Code                       | Con extensiones ESLint, Tailwind IntelliSense, GitLens. |

### 7.2 Instalación local

Pasos para arrancar el proyecto en local:

- `git clone https://github.com/AlbertoKimi/Visimap.git`
- `cd Visimap`
- `npm install` (descarga dependencias a `node_modules/`)
- Crear archivo `.env` con las claves de entorno (siguiente apartado)
- `npm run dev` (lanza servidor en `http://localhost:5173` con HMR)

### 7.3 Variables de entorno

Crea un archivo `.env` en la raíz del proyecto. Está en `.gitignore` y nunca se sube al repositorio:

| Variable               | Cómo obtenerla                                                |
|------------------------|---------------------------------------------------------------|
| VITE_SUPABASE_URL      | Panel de Supabase → Project Settings → API → Project URL.     |
| VITE_SUPABASE_ANON_KEY | Panel de Supabase → Project Settings → API → anon public key. |
| VITE_Groq_API_KEY      | console.groq.com → API Keys → Create Key.                     |

#### Prefijo VITE\_

Vite solo expone al navegador las variables que comienzan por `VITE_`. Cualquier variable sin ese prefijo es invisible para el código del cliente, lo que constituye una capa adicional de seguridad por defecto.

## 7.4 Comandos útiles

| Comando         | Descripción                                                         |
|-----------------|---------------------------------------------------------------------|
| npm install     | Descarga e instala todas las dependencias listadas en package.json. |
| npm run dev     | Lanza el servidor de Vite en modo desarrollo con HMR (puerto 5173). |
| npm run build   | Compila a la carpeta /dist optimizada para producción.              |
| npm run preview | Sirve localmente la última build de producción.                     |
| npm run lint    | Ejecuta ESLint sobre todo el proyecto.                              |

## 7.5 Despliegue en Vercel

Vercel es la plataforma de hosting elegida para Visimap. Ofrece despliegue continuo desde GitHub, CDN global, certificados SSL automáticos y previsualizaciones por cada Pull Request. Al ser una SPA, requiere una pequeña configuración (vercel.json) para que todas las rutas (/dashboard/mapa, /login...) sirvan el mismo index.html y dejen el enrutamiento al cliente.

- Importar el repositorio en vercel.com → New Project → Import Git Repository.
- Configuración detectada automáticamente: framework=Vite, build=npm run build, output=dist.
- Añadir las 3 variables VITE\_\* en Environment Variables del proyecto Vercel.
- Pulsar Deploy. Tras unos segundos la aplicación queda disponible en visimap.vercel.app.
- Despliegue continuo: cada push a main dispara un nuevo build automático.

## CAPÍTULO 08 · Esquemas de Flujo

En este capítulo se presentan de forma visual los flujos clave del sistema: la arquitectura general de Visimap (interacción entre el cliente React y los servicios de Supabase) y los dos diagramas de secuencia que describen los procesos críticos de la aplicación: la autenticación de usuarios y el registro de visitas en el mapa interactivo.

### 8.1 Arquitectura general

Visimap adopta una arquitectura Front-only: toda la lógica de presentación reside en el navegador y delega la persistencia, autenticación y APIs al BaaS de Supabase. El siguiente diagrama muestra cómo el cliente React+Vite se comunica, a través del SDK de Supabase, con los tres servicios principales: PostgreSQL, Supabase Auth y la API REST PostgREST.

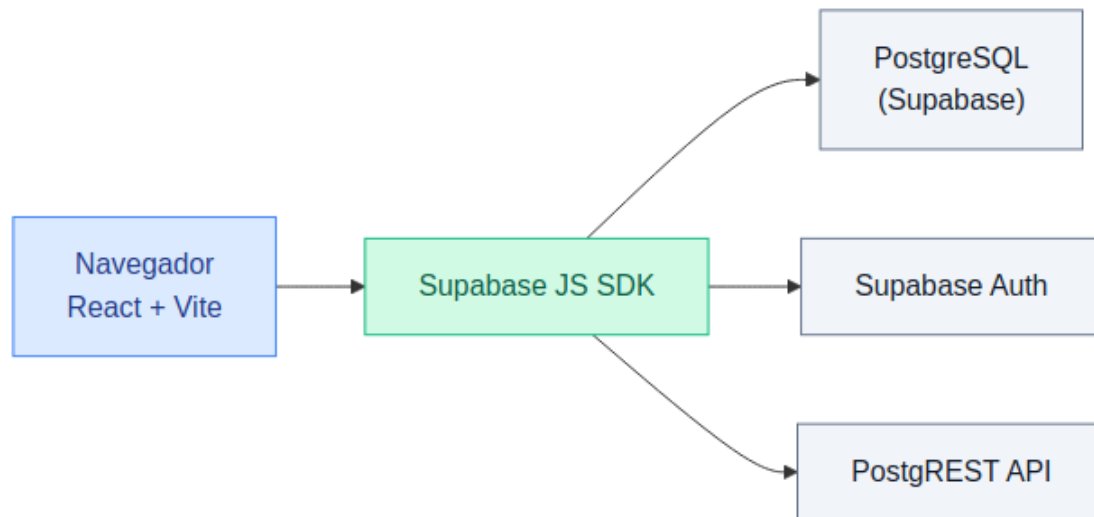


Figura 8.1 — Arquitectura Front-only de Visimap

### 8.2 Diagrama de secuencia · Flujo de Autenticación

El flujo de autenticación inicia con el usuario introduciendo email y contraseña. La aplicación React invoca `supabase.auth.signInWithPassword()`, que valida las credenciales contra la tabla `auth.users` de PostgreSQL. Cuando son correctas, Supabase devuelve un objeto `session` con el `access_token` y los datos del usuario. El `AuthContext` de React guarda esta sesión y redirige al usuario a la ruta `/dashboard`.

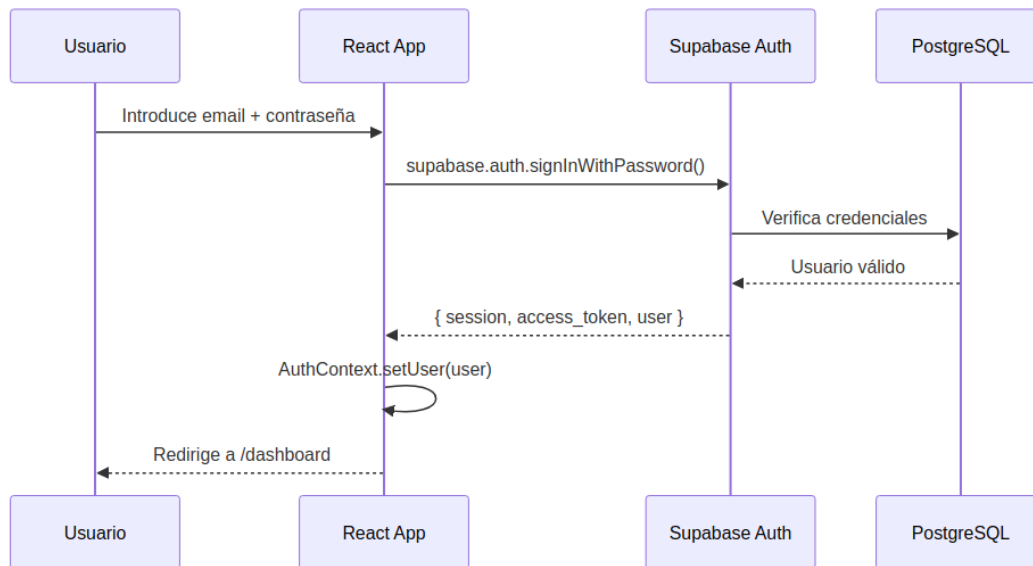


Figura 8.2 — Flujo de autenticación con Supabase Auth

### 8.3 Diagrama de secuencia · Registro de Visita

Cuando un trabajador hace clic en una provincia del mapa SVG (componente SpainMap), se abre un modal de registro. Tras introducir el número de visitantes, el componente llama al hook useVisitors, que ejecuta un INSERT contra la tabla visitors mediante el cliente de Supabase. La respuesta confirma el registro, se actualiza el estado local y se muestra una notificación toast al usuario.

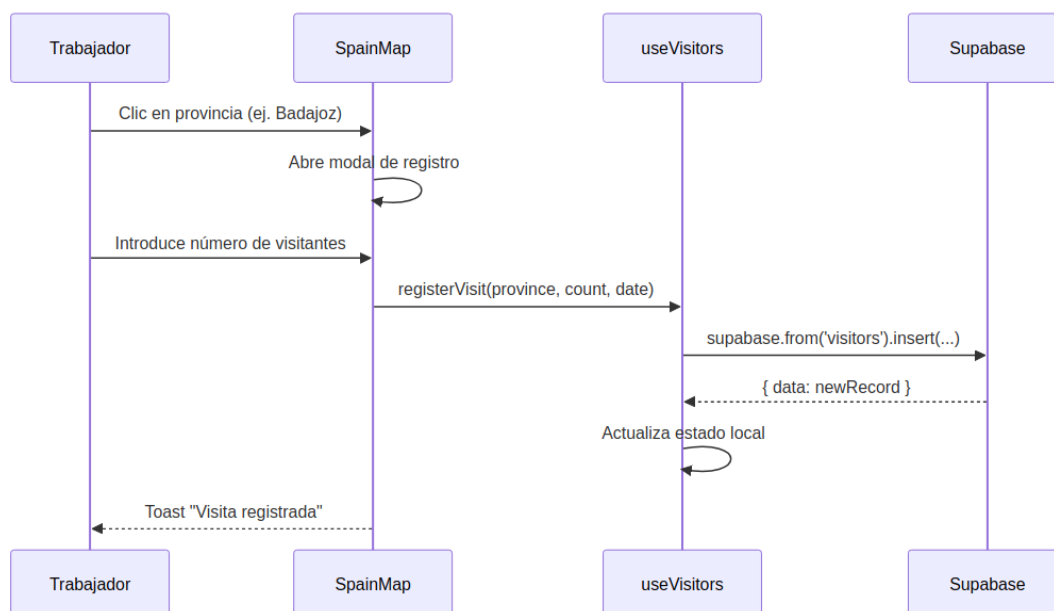


Figura 8.3 — Flujo de registro de visita en el mapa interactivo

## 8.4 Diagrama de secuencia · Creación de Evento en Calendario

Cuando un empleado hace clic sobre una franja del calendario (componente VistaCalendario, basado en FullCalendar), se abre el modal EventoModal con la fecha pre-rellenada. Tras introducir el nombre, el tipo de evento y las fechas, el componente invoca al SupabaseEventRepository, que lanza un INSERT contra la tabla evento mediante PostgREST. Las políticas RLS validan que el usuario esté autenticado y activo. La respuesta confirma el registro, se cierra el modal, FullCalendar se refresca y se muestra una notificación toast al usuario.

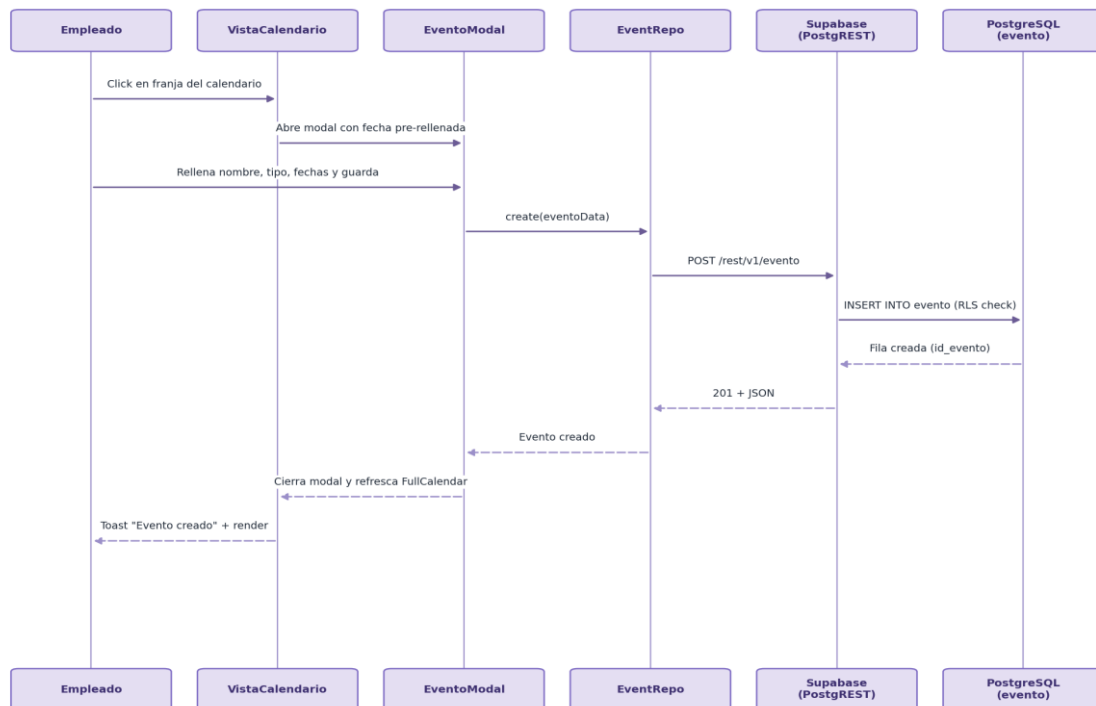


Figura 8.4 — Flujo de creación de un evento en el calendario

## 8.5 Diagrama de secuencia · Invitación de Empleado (Edge Function)

Desde la VistaUsuarios, el administrador rellena el formulario de invitación. El SupabaseAuthRepository invoca la Edge Function invite-user mediante `supabase.functions.invoke()`. La Edge Function, que se ejecuta en el entorno serverless de Deno con la `service_role` key como secreto, llama a `admin.inviteUserByEmail()` del SDK admin de Supabase. Supabase Auth genera un magic link y envía el correo. Cuando el empleado pulsa el enlace, aterriza en la página `/establecer-contrasena`, donde define su contraseña por primera vez. Este patrón garantiza que la `service_role` key nunca quede expuesta en el navegador.

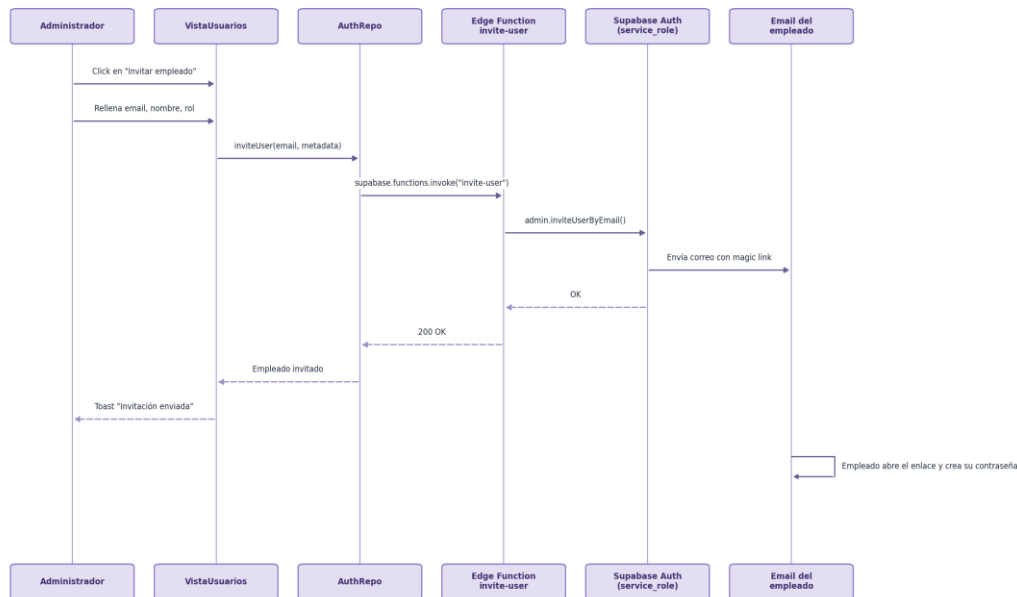


Figura 8.5 — Invitación de un nuevo empleado mediante la Edge Function invite-user

## 8.6 Diagrama de secuencia · Asistente IA

El usuario formula una pregunta en lenguaje natural en la página /dashboard/asistente. El hook useChatIA delega en openaiService, que envía la pregunta al modelo de Groq junto con un system prompt que incluye el esquema de la base de datos y las vistas SQL. El modelo responde con un bloque especial ``db-query`` que el cliente parsea y ejecuta de forma controlada contra Supabase respetando las políticas RLS. El resultado se devuelve al modelo en una segunda llamada; este genera la respuesta final en texto natural y, opcionalmente, un bloque ``chart`` con la configuración de un gráfico de Recharts. El componente ChatIA renderiza el texto y, si procede, dibuja el gráfico.

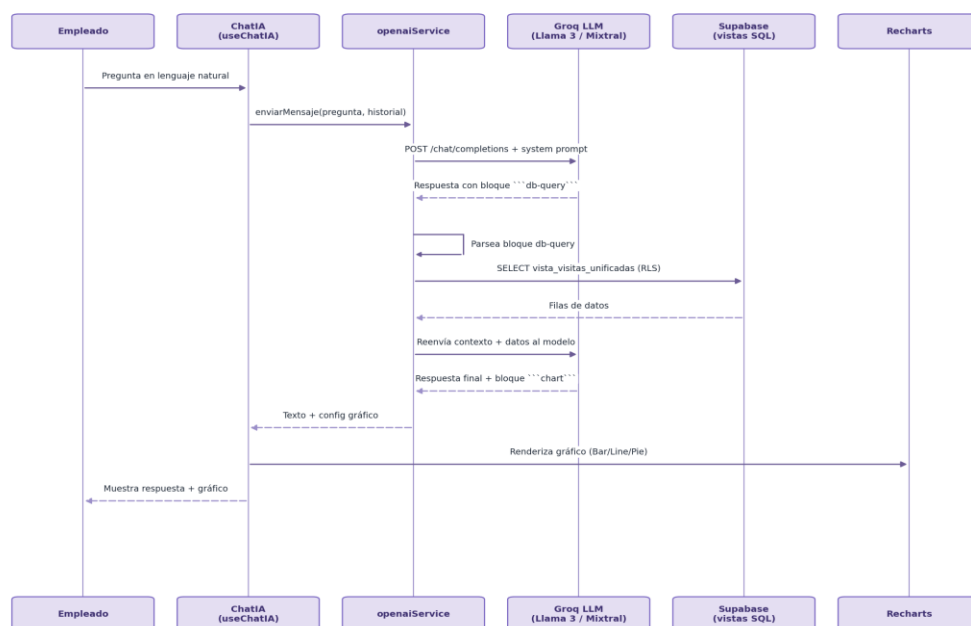


Figura 8.6 — Flujo conversacional del asistente IA con consulta a BD y generación de gráfico

## 8.7 Diagrama de secuencia · Exportación de Histórico Anual a PDF

El administrador selecciona un año en la VistaHistorial y pulsa “Exportar PDF”. El VisitorRepository consulta la vista vista\_visitantes\_totales para obtener los datos agregados por mes, provincia y país. La vista está creada con security\_invoker para que las políticas RLS sigan aplicándose. La librería html2canvas captura los nodos del DOM con las gráficas de Recharts y los convierte en imágenes PNG. Finalmente, jsPDF junto con jsPDF-AutoTable construyen el documento en cliente: tabla de totales, imágenes embebidas, encabezado y pie. El navegador descarga el archivo historico-AÑO.pdf sin necesidad de servidor.

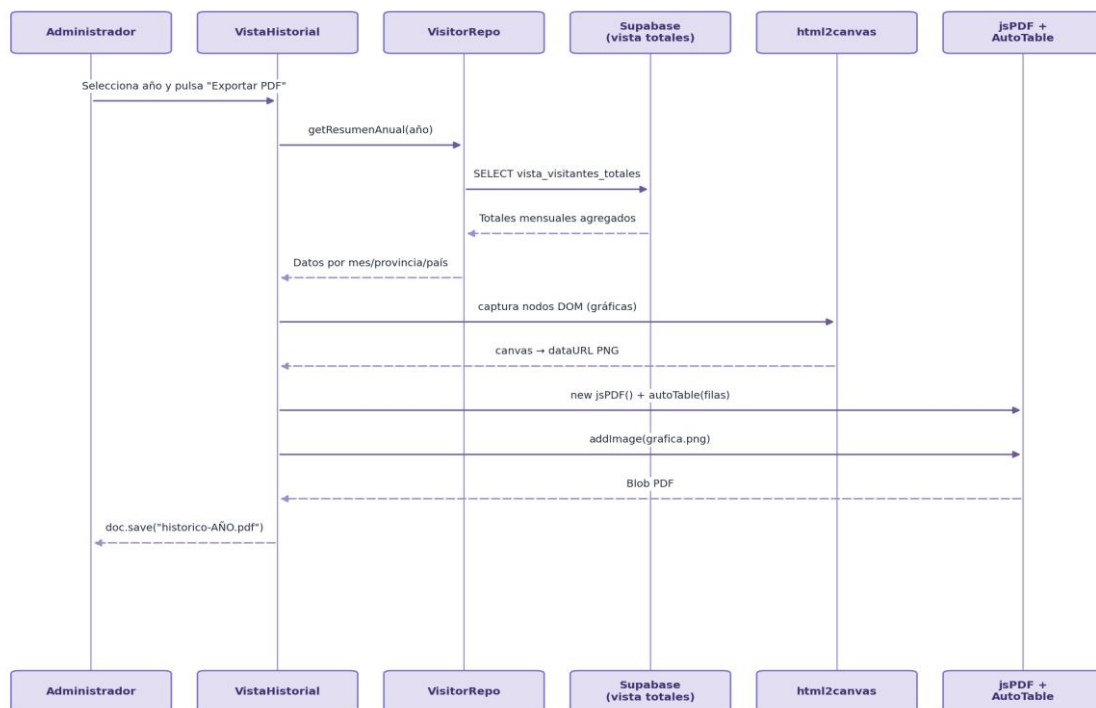


Figura 8.7 — Exportación del histórico anual a PDF mediante jsPDF y html2canvas

## 8.8 Diagrama de secuencia · Sistema de Notas

El sistema de notas tipo post-it permite crear, asignar y cambiar el estado de recordatorios entre los empleados del museo. Al cargar la página, el repositorio recupera las notas recientes y las filtra según el rol y la asignación del usuario. La creación de una nueva nota abre un modal donde se elige destinatario, título y contenido; tras confirmar, el registro se inserta en Supabase y la vista se refresca automáticamente.

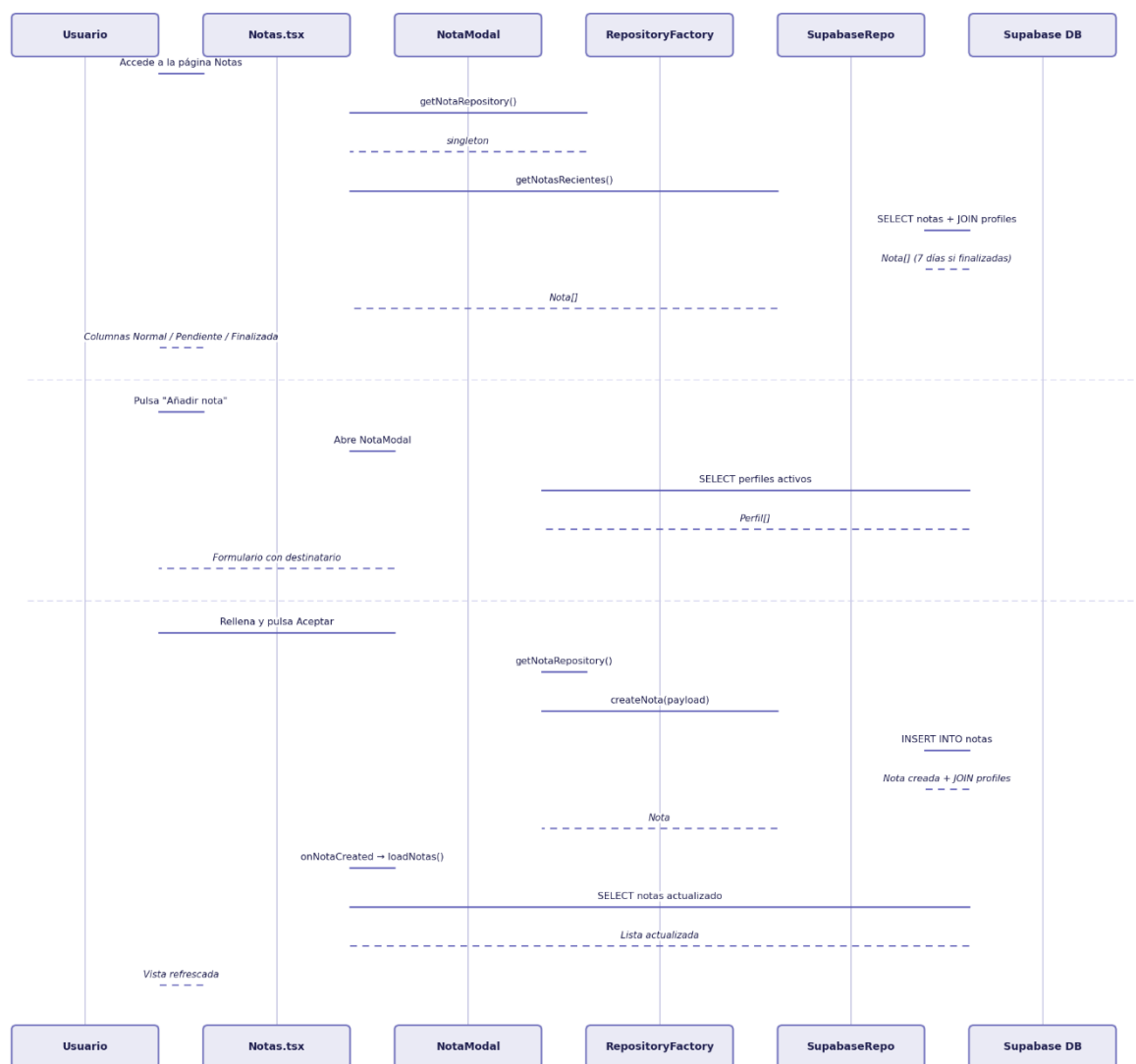


Figura 8.8 — Flujo de carga y creación de notas internas

## 8.9 Diagrama de secuencia · Notificaciones

El sistema de notificaciones permite que los empleados estén al tanto de la actividad reciente del museo sin necesidad de revisar manualmente cada sección de la aplicación. El ícono de campana en la cabecera muestra un punto rojo cuando hay avisos nuevos pendientes de leer.

Las alertas se generan a partir de dos tipos de actividad: **notas** y **eventos**. Cuando un compañero crea una nota o la marca como finalizada, el destinatario recibe un aviso. En el caso de los eventos del calendario, el sistema notifica cuando un evento ha sido creado por otro empleado, cuando está a punto de comenzar, cuando su fecha de finalización ha llegado y está pendiente de confirmación, o cuando se confirma como finalizado.

Para detectar esta actividad, la campana consulta automáticamente la base de datos cada 30 segundos en segundo plano. Las alertas obtenidas se filtran según el rol y la asignación de cada usuario, de modo que cada empleado solo ve lo que le corresponde. El sistema compara las alertas

recibidas con un registro local de lo que ya ha leído, y solo muestra el punto rojo cuando hay algo nuevo.

Al pulsar la campana se despliega el panel con los avisos de los últimos tres días, ordenados del más reciente al más antiguo. En cuanto el empleado cierra el panel, todas las alertas visibles quedan marcadas como leídas y el punto rojo desaparece hasta que llegue un nuevo aviso.

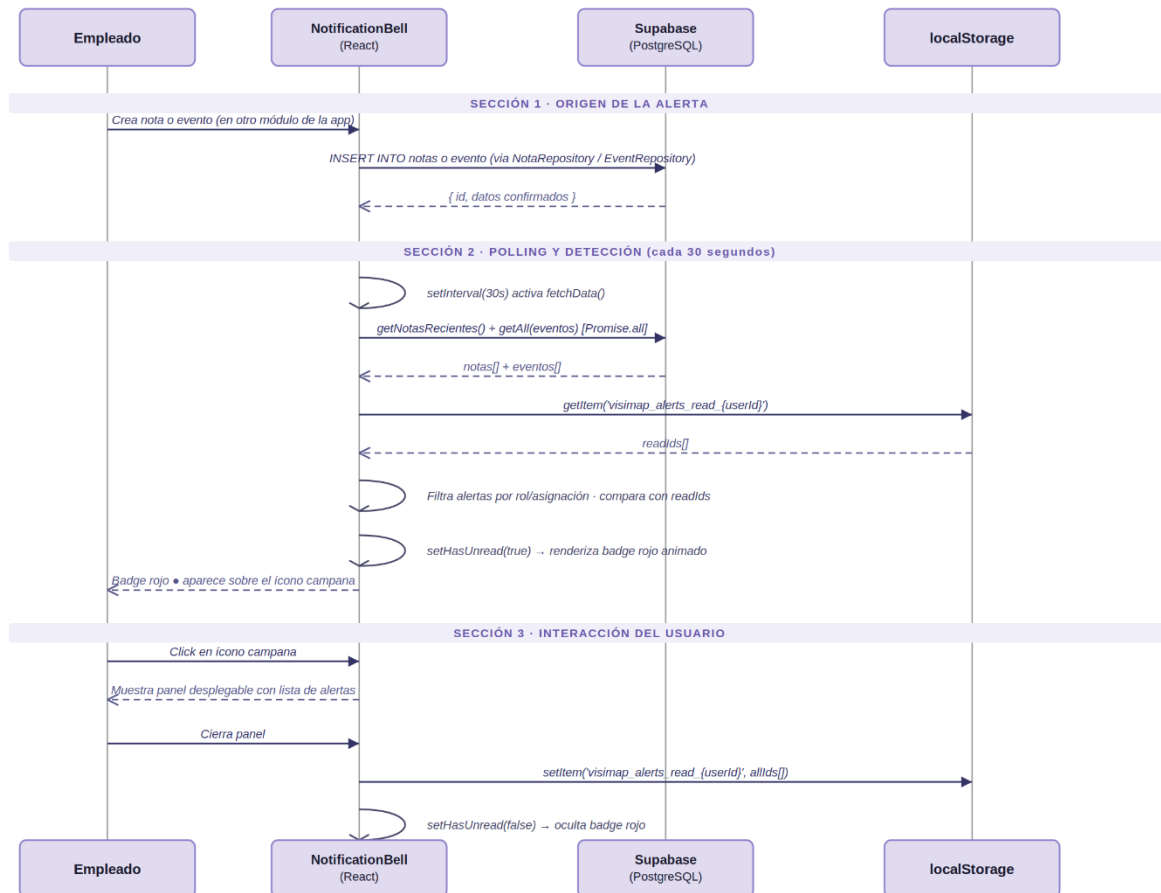


Figura 8.9 — Flujo de generación y visualización de las notificaciones (campana)

## CAPÍTULO 09 · Pruebas Realizadas

Se ha llevado a cabo una batería de pruebas para verificar el correcto funcionamiento de Visimap a tres niveles: pruebas funcionales sobre los flujos principales, pruebas de seguridad sobre las políticas RLS de PostgreSQL, y pruebas de rendimiento del bundle de producción y del Lighthouse Score.

### 9.1 Pruebas Funcionales

Las pruebas funcionales cubren los casos de uso principales del sistema, incluyendo autenticación, registro de visitas, visualización de estadísticas y control de acceso por rol. Cada prueba define la entrada, el resultado esperado y el resultado obtenido.

| ID   | Caso de prueba                      | Entrada                                | Esperado                                | Obtenido                               | Estado |
|------|-------------------------------------|----------------------------------------|-----------------------------------------|----------------------------------------|--------|
| P-01 | Login con credenciales válidas      | Email y contraseña correctos           | Redirección a /dashboard                | Redirección a /dashboard               | OK     |
| P-02 | Login con contraseña incorrecta     | Contraseña errónea                     | Mensaje de error                        | Mensaje de error de Supabase           | OK     |
| P-03 | Registro de visita en mapa          | Clic en Badajoz, count=15              | Registro insertado en BD                | Registro insertado                     | OK     |
| P-04 | Visualización estadísticas          | Mes: Enero 2025                        | Gráfica con datos                       | Gráfica renderizada                    | OK     |
| P-05 | Acceso a admin desde rol trabajador | Ruta /admin                            | Redirección a /dashboard                | Redirección funcionando                | OK     |
| P-06 | Cierre de sesión                    | Clic en logout                         | Sesión eliminada                        | Sesión eliminada                       | OK     |
| P-07 | Registro visitante internacional    | País: Francia, count=8                 | Registro insertado en BD                | Registro insertado                     | OK     |
| P-08 | Eliminación de registro (admin)     | Delete en historial                    | Registro eliminado                      | Registro eliminado y lista actualizada | OK     |
| P-09 | Crear evento en calendario          | Nombre, tipo y rango de fechas válidos | Evento creado y visible en FullCalendar | Evento renderizado y persistido        | OK     |
| P-10 | Editar evento existente             | Click en evento y modificar nombre     | Cambios guardados en BD                 | UPDATE correcto y refresco visual      | OK     |

|      |                                             |                                        |                                                        |                                          |    |
|------|---------------------------------------------|----------------------------------------|--------------------------------------------------------|------------------------------------------|----|
| P-11 | Eliminar evento                             | Click en eliminar dentro del modal     | Evento borrado de la BD y del calendario               | DELETE con cascade en grupo_visitante    | OK |
| P-12 | Asociar grupo de visitantes a evento        | Provincia: Cáceres, num=25             | Fila insertada en grupo_visitante                      | Grupo asociado al id_evento              | OK |
| P-13 | Crear nota interna                          | Título y contenido válidos             | Nota creada con estado=normal                          | Nota visible en /dashboard/notas         | OK |
| P-14 | Asignar nota a otro empleado                | Seleccionar destinatario en el modal   | Campo asignado_a actualizado                           | Nota aparece en bandeja del destinatario | OK |
| P-15 | Cambiar estado de nota a finalizada         | Click en “Marcar como finalizada”      | estado='finalizada' y actualizado_en=now()             | Estado actualizado correctamente         | OK |
| P-16 | Pregunta de texto al asistente IA           | “¿Cuántos visitantes en julio?”        | Respuesta con cifra exacta                             | IA ejecuta db-query y responde           | OK |
| P-17 | Generación de gráfico por la IA             | “Muéstrame un gráfico mensual de 2025” | Bloque chart + render Recharts                         | Gráfico de barras renderizado            | OK |
| P-18 | Invitación de nuevo empleado (admin)        | Email válido + rol trabajador          | Edge Function envía magic link                         | Correo recibido y enlace operativo       | OK |
| P-19 | Desactivar empleado                         | Toggle active=false en VistaUsuarios   | Empleado no puede iniciar sesión                       | fetchProfile() fuerza signOut()          | OK |
| P-20 | Cambio de contraseña desde perfil           | Nueva contraseña válida                | supabase.auth.updateUser() OK                          | Contraseña actualizada                   | OK |
| P-21 | Subida de avatar en perfil                  | Imagen JPG de 400 KB                   | Archivo en bucket avatars y URL en profiles.avatar_url | Avatar visible en sidebar y header       | OK |
| P-22 | Exportar histórico anual a PDF              | Año 2025 + click “Exportar PDF”        | Archivo historico-2025.pdf descargado                  | PDF generado con tabla y gráficas        | OK |
| P-23 | Acceso al asistente IA desde rol trabajador | URL /dashboard/asistente               | Redirección o vista bloqueada                          | Menú oculto y ruta bloqueada             | OK |
| P-24 | Cambio de tema claro/oscuro                 | Click en ThemeToggle                   | Clase 'dark' aplicada y persistencia                   | Tema persistido entre sesiones           | OK |

## 9.2 Pruebas de Seguridad (Row Level Security)

Las pruebas de seguridad verifican que las políticas RLS de PostgreSQL bloquean correctamente cualquier acceso no autorizado, incluso si un atacante interceptase la anon key e intentara saltarse el frontend.

| Prueba                                      | Resultado                     |
|---------------------------------------------|-------------------------------|
| Acceso a tabla visitors sin sesión activa   | Bloqueado por RLS             |
| INSERT con token expirado                   | Error 401 Unauthorized        |
| Trabajador intenta DELETE en tabla visitors | Bloqueado por política RLS    |
| Acceso directo a URL /admin sin rol admin   | Redirección en ProtectedRoute |

## 9.3 Pruebas de Rendimiento

Las pruebas de rendimiento se realizaron sobre el bundle de producción desplegado en Vercel utilizando Lighthouse de Chrome DevTools. Los resultados confirman la efectividad de las optimizaciones de Vite (code splitting, tree shaking) y de Tailwind (purga de CSS no utilizado).

| Métrica                     | Resultado                         |
|-----------------------------|-----------------------------------|
| Lighthouse · Performance    | 98 / 100                          |
| Lighthouse · Accessibility  | 96 / 100                          |
| Lighthouse · Best Practices | 100 / 100                         |
| Lighthouse · SEO            | 92 / 100                          |
| Bundle JS total (gzipped)   | ~280 KB                           |
| Vendor chunk                | Separado del código de aplicación |
| CSS purgado en producción   | 18 KB                             |

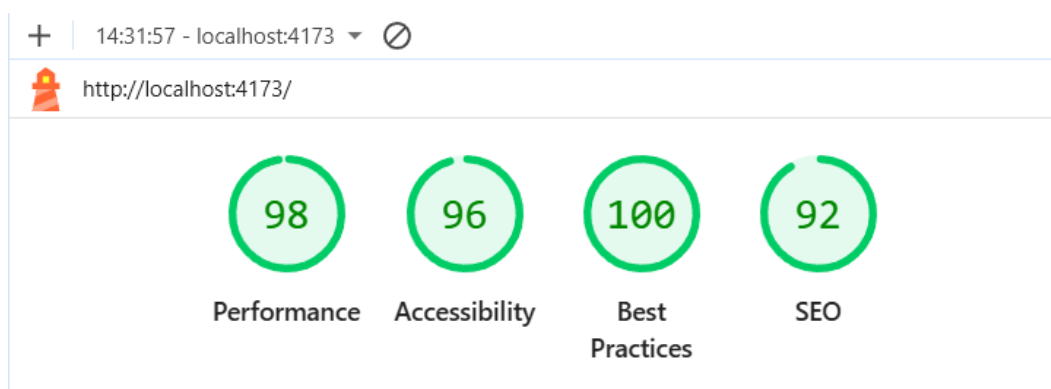


Figura 9.a — Datos de LightHose de Visimap en PC

## CAPÍTULO 10 · Conclusión e Implementaciones Futuras

### Resumen:

Visimap combina arquitectura moderna, buenas prácticas de ingeniería y resolución de un problema real. La migración a TypeScript realizada tras la primera tutoría, junto con la introducción del patrón Repository y la refactorización de la jerarquía de carpetas, han elevado significativamente la calidad del código respecto al prototipo inicial en JavaScript.

A nivel funcional, el sistema ya cubre la totalidad la mayoría de los requisitos planteados por el museo MUVI: registro geográfico de visitantes, gestión de eventos, comunicación interna, panel administrativo de personal, dashboard analítico, histórico anual con exportación a PDF y asistente IA en página dedicada. La interfaz responsive permite operar desde el ordenador en el Muvi, o en una Tablet o desde tu teléfono móvil en cualquier parte por si quieres revisar los registros o notas que te pueden dejar tus empleados, etc. El despliegue en Vercel hace que la herramienta esté disponible 24/7 sin coste de infraestructura.

### Implementaciones o mejoras futuras

| Mejora               | Descripción                                                                                                                                                                                                              |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Internacionalización | Soporte multiidioma (ES/EN/PT) usando react-i18next, especialmente útil para los visitantes internacionales y para mostrar la landing en varios idiomas.                                                                 |
| Landing del Muvi     | Convertir la landing page de visimap en la del propio museo.                                                                                                                                                             |
| Registro de usuarios | Los usuarios se podrán registrar en la página del museo.                                                                                                                                                                 |
| Zona privada         | Una vez registrado, el usuario accederá a una zona privada para ver las noticias del museo, próximas exposiciones, zona para reservar visita guiada, zona para entrega de documentación, apuntarse a una actividad, etc. |
| Mejorar la IA        | Mover las llamadas a Claude (Anthropic) a una Edge Function de Supabase para ocultar la API key y permitir caché de respuestas frecuentes.                                                                               |
| Motor de reservas    | Integración para reservas online de talleres, exposiciones con aforo limitado, visitas guiadas, etc.                                                                                                                     |
| Correo               | Implementación de una zona de correo del museo para que no tenga que ir a otro sistema de correo y lo tenga todo a mano en visimap                                                                                       |
| Posicionamiento SEO  | Implementación de la landing page en Astro para el posicionamiento SEO del Muvi.                                                                                                                                         |
| Rendimiento          | Mejora del rendimiento del aplicativo que afecta también en el posicionamiento SEO.                                                                                                                                      |